

SRS - StudyGroup



Use Case Descriptions

UC-1 User Registration

Primary Actor:

Guest User

Description:

Allows a new user to create an account as Student or Creator.

Preconditions:

* User is not registered before.

Main Flow:

1. User enters registration data.
2. System validates the input.
3. System creates the account.
4. If role is Creator, creator request is marked as pending approval.
5. System returns success message.

Postconditions:

* User account is created successfully.

UC-2 User Login

Primary Actor:

Registered User

Description:

Allows users to log into the system.

Preconditions:

* User account exists.

Main Flow:

1. User enters email and password.
2. System validates credentials.
3. System generates JWT token.
4. System returns authentication token.

Postconditions:

* User becomes authenticated.

UC-3 Approve Creator Account

Primary Actor:

Admin

Description:

Allows admin to approve creator accounts.

Preconditions:

* Creator request exists and is pending.

Main Flow:

1. Admin views pending creator requests.
2. Admin selects a request.
3. System updates creator status to approved.

Postconditions:

* Creator can access creator functionalities.

UC-4 Reject Creator Account

Primary Actor:

Admin

Description:

Allows admin to reject creator accounts.

Preconditions:

* Creator request exists and is pending.

Main Flow:

1. Admin views creator requests.
2. Admin rejects selected request.
3. System updates creator status.

Postconditions:

* Creator account becomes rejected.

UC-5 Create Study Group

Primary Actor:

Creator

Description:

Allows approved creators to create study groups.

Preconditions:

* Creator account is approved.

Main Flow:

1. Creator enters group information.
2. System validates data.
3. System stores group with pending status.

Postconditions:

* Group request is created successfully.

UC-6 Approve Study Group

Primary Actor:

Admin

Description:

Allows admin to approve study groups.

Preconditions:

* Group exists with pending status.

Main Flow:

1. Admin views pending groups.
2. Admin approves selected group.
3. System updates group status.
4. System adds creator as first group member.

Postconditions:

* Group becomes available to users.

UC-7 Reject Study Group

Primary Actor:

Admin

Description:

Allows admin to reject study groups.

Preconditions:

* Group exists with pending status.

Main Flow:

1. Admin selects group request.
2. Admin rejects the group.
3. System updates group status.

Postconditions:

* Group becomes rejected.

UC-8 View Study Groups

Primary Actor:

User

Description:

Allows users to view approved study groups.

Preconditions:

* Approved groups exist.

Main Flow:

1. User requests group list.
2. System retrieves approved groups.
3. System displays groups.

Postconditions:

* User views available groups.

UC-9 Search Study Groups

Primary Actor:

User

Description:

Allows users to search for study groups.

Preconditions:

* Approved groups exist.

Main Flow:

1. User enters search criteria.
2. System filters groups.
3. System displays matching groups.

Postconditions:

* Matching groups are displayed.

UC-10 View Group Details

Primary Actor:

User

Description:

Allows users to view detailed information about a group.

Preconditions:

* Group exists and is approved.

Main Flow:

1. User selects a group.
2. System retrieves group details.
3. System displays details.

Postconditions:

* Group information is displayed.

UC-11 Update Group

Primary Actor:

Creator

Description:

Allows creators to update their groups.

Preconditions:

* Creator owns the group.

Main Flow:

1. Creator edits group data.

2. System validates updates.

3. System saves changes.

Postconditions:

* Group information is updated.

UC-12 Delete Group

Primary Actor:

Creator

Description:

Allows creators to delete their groups.

Preconditions:

* Creator owns the group.

Main Flow:

1. Creator selects group.

2. System deletes the group.

Postconditions:

* Group is removed from the system.

UC-13 Send Join Request

Primary Actor:

Student

Description:

Allows students to request joining a group.

Preconditions:

* Group exists.

* Student is not already a member.

Main Flow:

1. Student selects group.

2. System creates join request.

3. Request status becomes pending.

Postconditions:

* Join request is submitted.

UC-14 View Join Requests

Primary Actor:

Creator

Description:

Allows creators to view pending join requests.

Preconditions:

* Creator has groups.

Main Flow:

1. Creator opens requests page.
2. System retrieves pending requests.
3. System displays requests.

Postconditions:

* Requests are displayed successfully.

UC-15 Accept Join Request

Primary Actor:

Creator

Description:

Allows creators to accept student requests.

Preconditions:

* Request exists and is pending.

Main Flow:

1. Creator selects request.
2. System updates request status.
3. Student is added as group member.

Postconditions:

* Student joins the group.

UC-16 Reject Join Request

Primary Actor:

Creator

Description:

Allows creators to reject student requests.

Preconditions:

* Request exists and is pending.

Main Flow:

1. Creator selects request.
2. System updates request status to rejected.

Postconditions:

* Request becomes rejected.

UC-17 View Joined Groups

Primary Actor:

Student

Description:

Allows students to view groups they joined.

Preconditions:

- * Student joined at least one group.

Main Flow:

1. Student opens joined groups page.
2. System retrieves joined groups.
3. System displays groups.

Postconditions:

- * Student views joined groups.

UC-18 Upload Study Material

Primary Actor:

Group Member

Description:

Allows members to upload study materials.

Preconditions:

- * User is a group member.

Main Flow:

1. User uploads file and description.
2. System validates membership.
3. System stores material.

Postconditions:

- * Material is uploaded successfully.

UC-19 View Study Materials

Primary Actor:

Group Member

Description:

Allows members to view uploaded materials.

Preconditions:

- * Group exists.

Main Flow:

1. User opens materials page.
2. System retrieves materials.
3. System displays uploaded files.

Postconditions:

- * Materials are displayed.

UC-20 Add Comment

Primary Actor:

Group Member

Description:

Allows members to comment on materials.

Preconditions:

* User belongs to the material's group.

Main Flow:

1. User enters comment.
2. System validates membership.
3. System stores comment.

Postconditions:

* Comment is added successfully.

UC-21 View Comments

Primary Actor:

User

Description:

Allows users to view comments on materials.

Preconditions:

* Material exists.

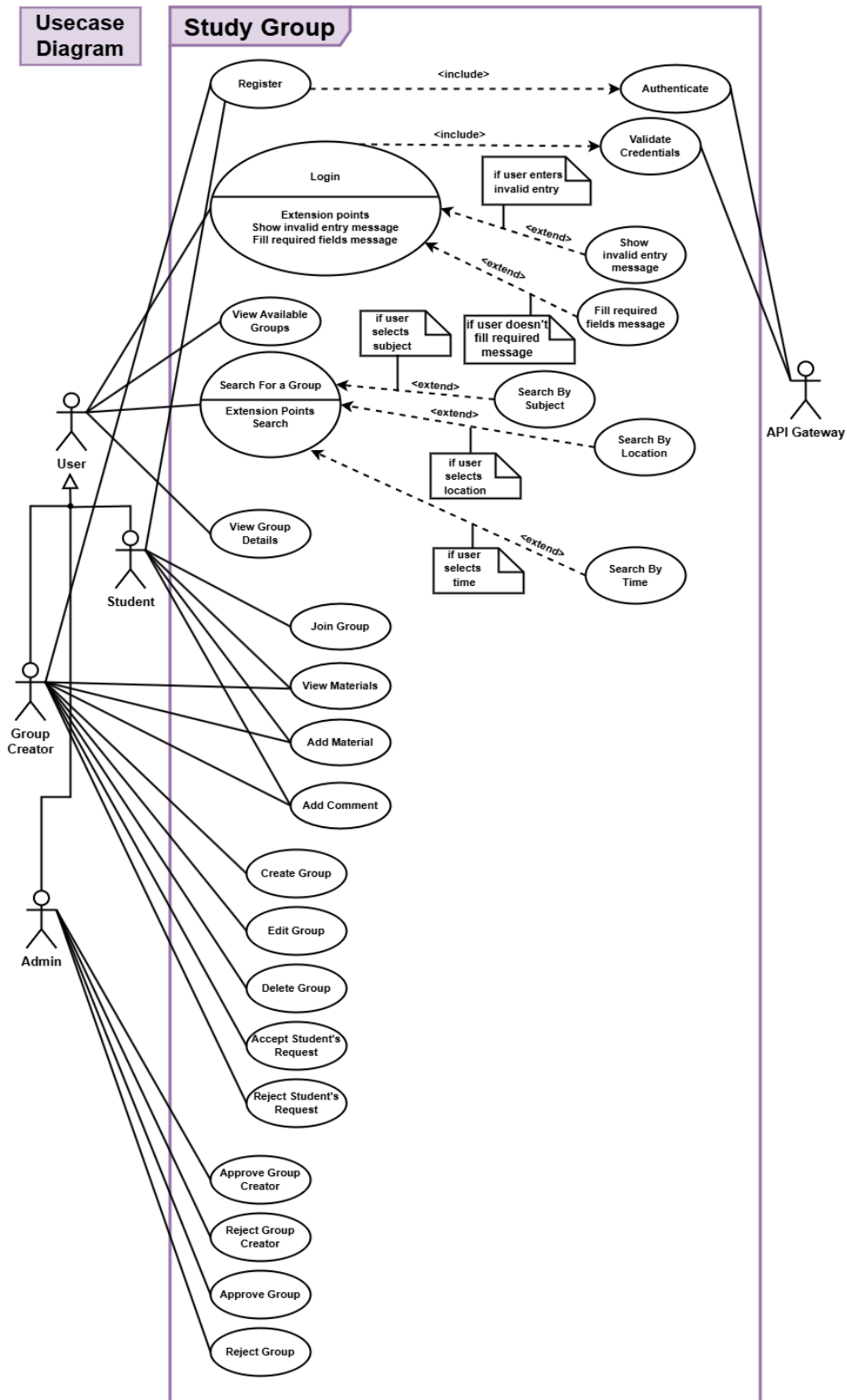
Main Flow:

1. User opens comments section.
2. System retrieves comments.
3. System displays comments.

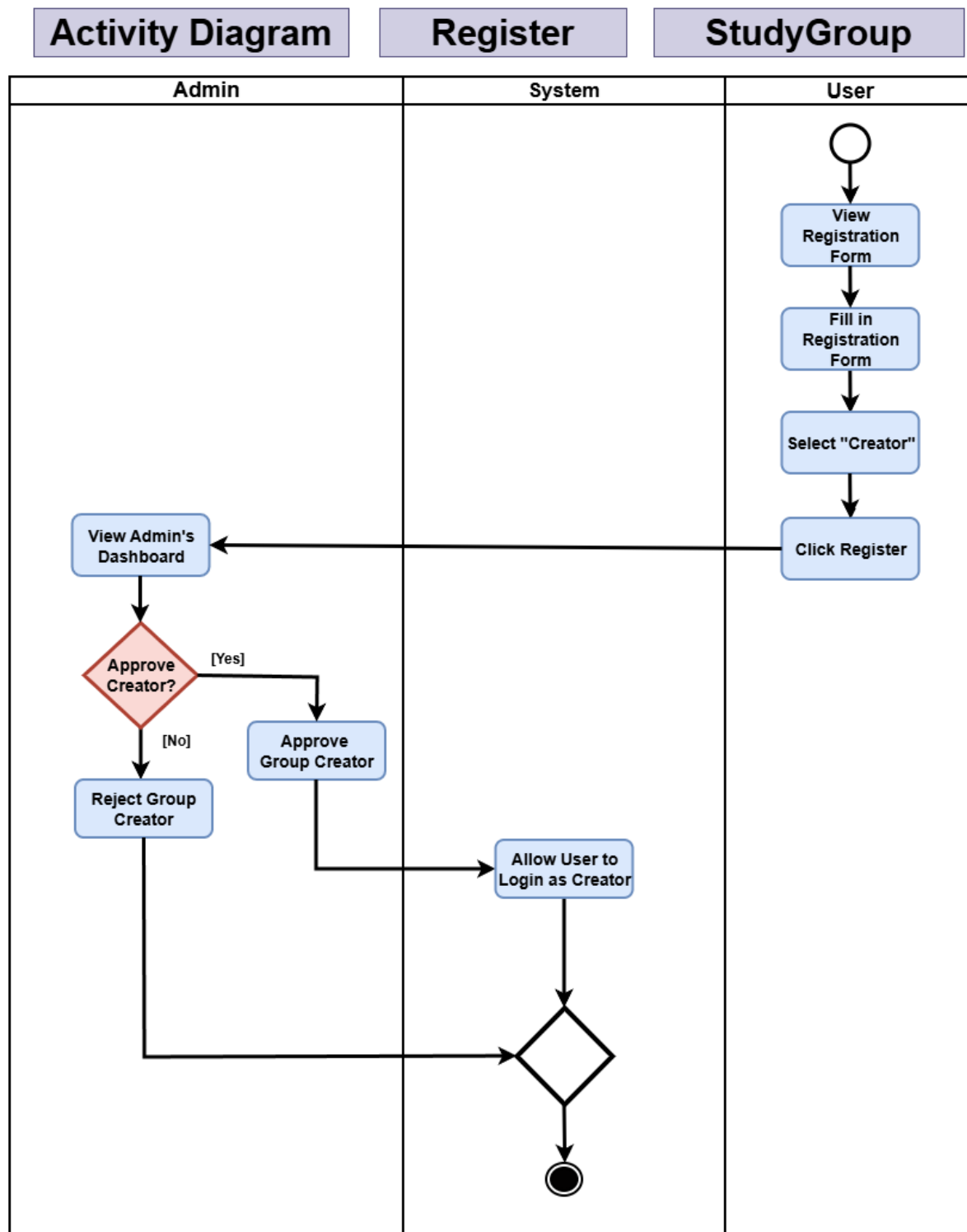
Postconditions:

* Comments are displayed successfully.

Usecase Diagram:



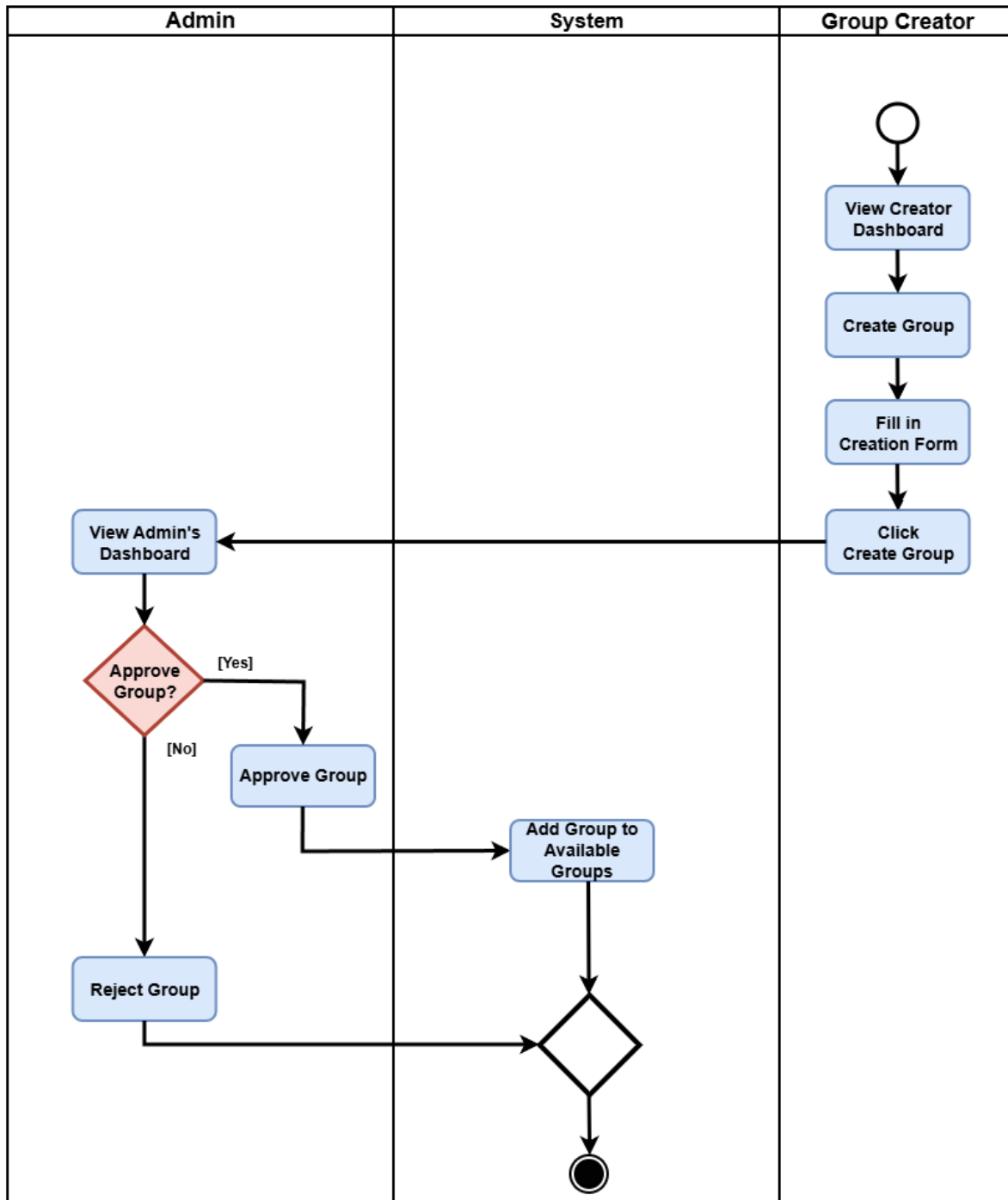
Activity Diagrams:



Activity Diagram

Create Group

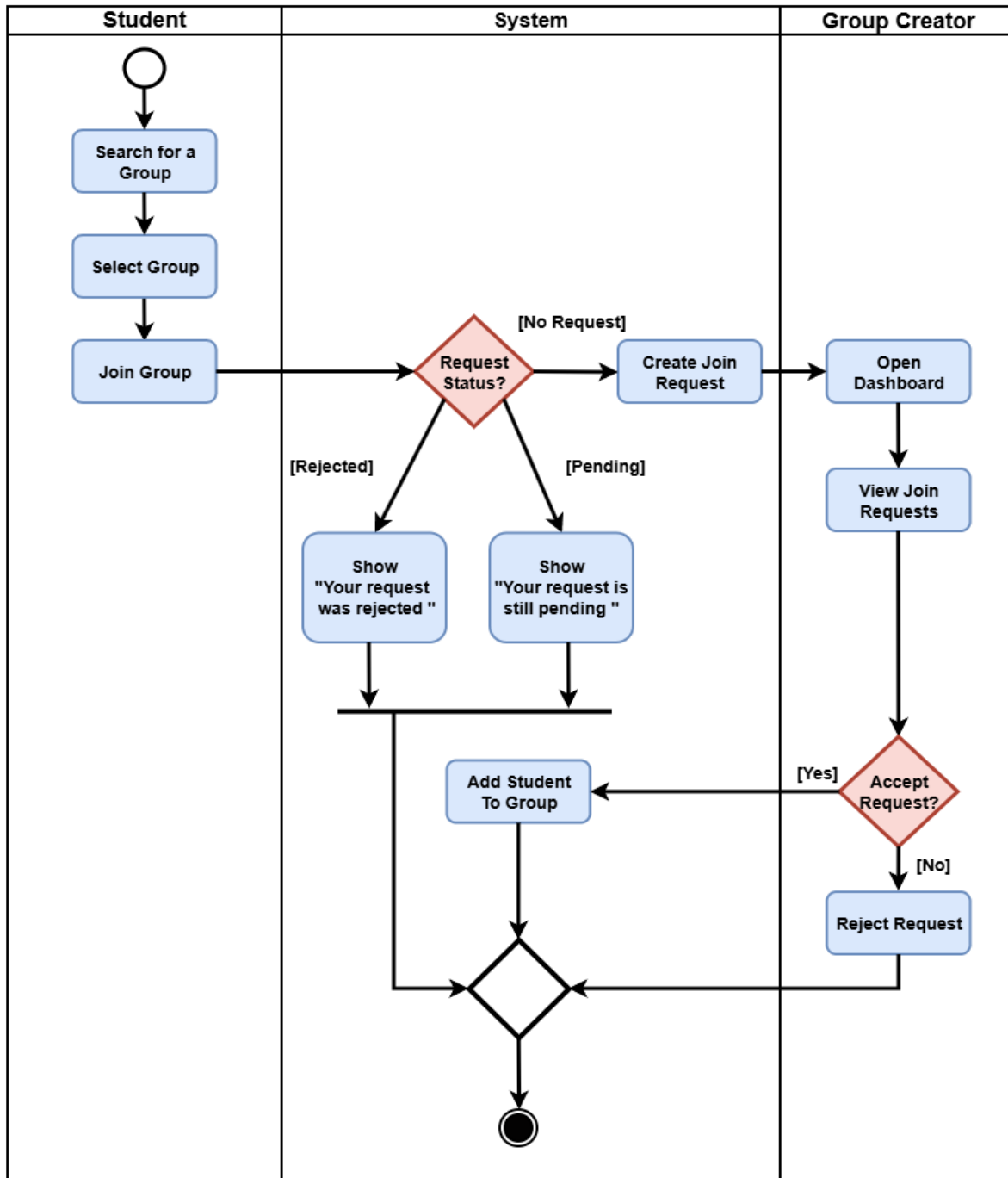
StudyGroup



Activity Diagram

Join Group

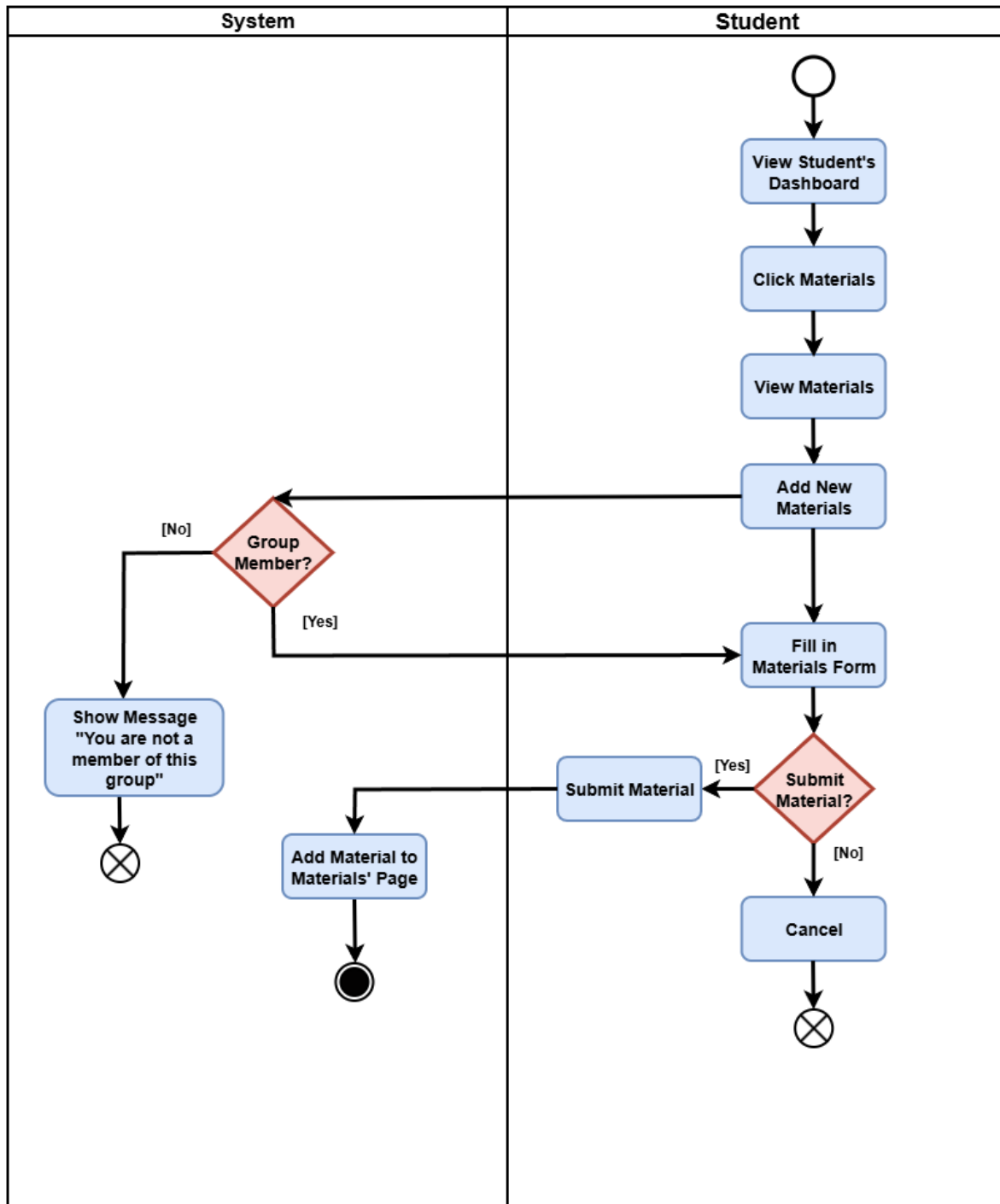
StudyGroup



Activity Diagram

Add Material

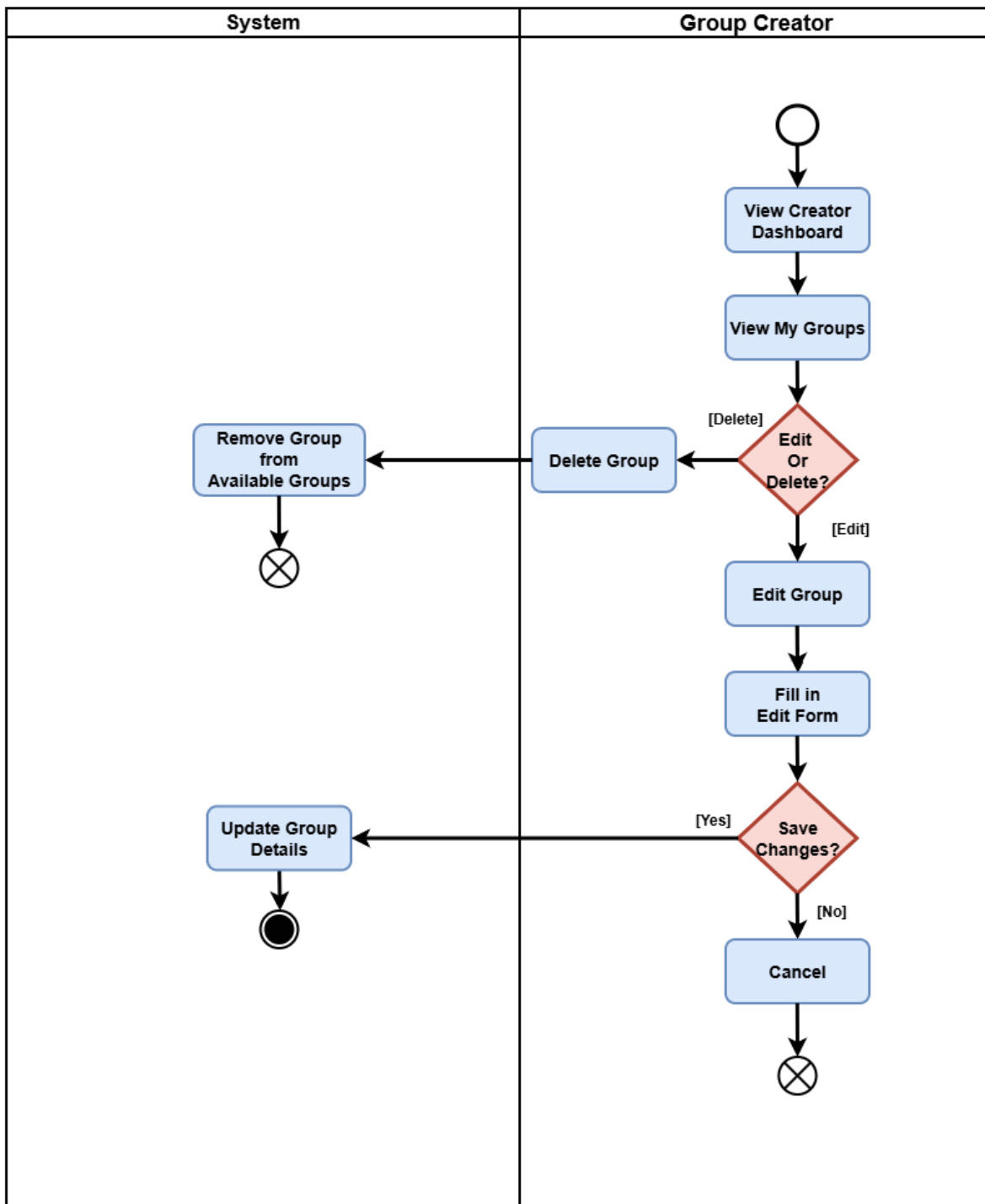
StudyGroup



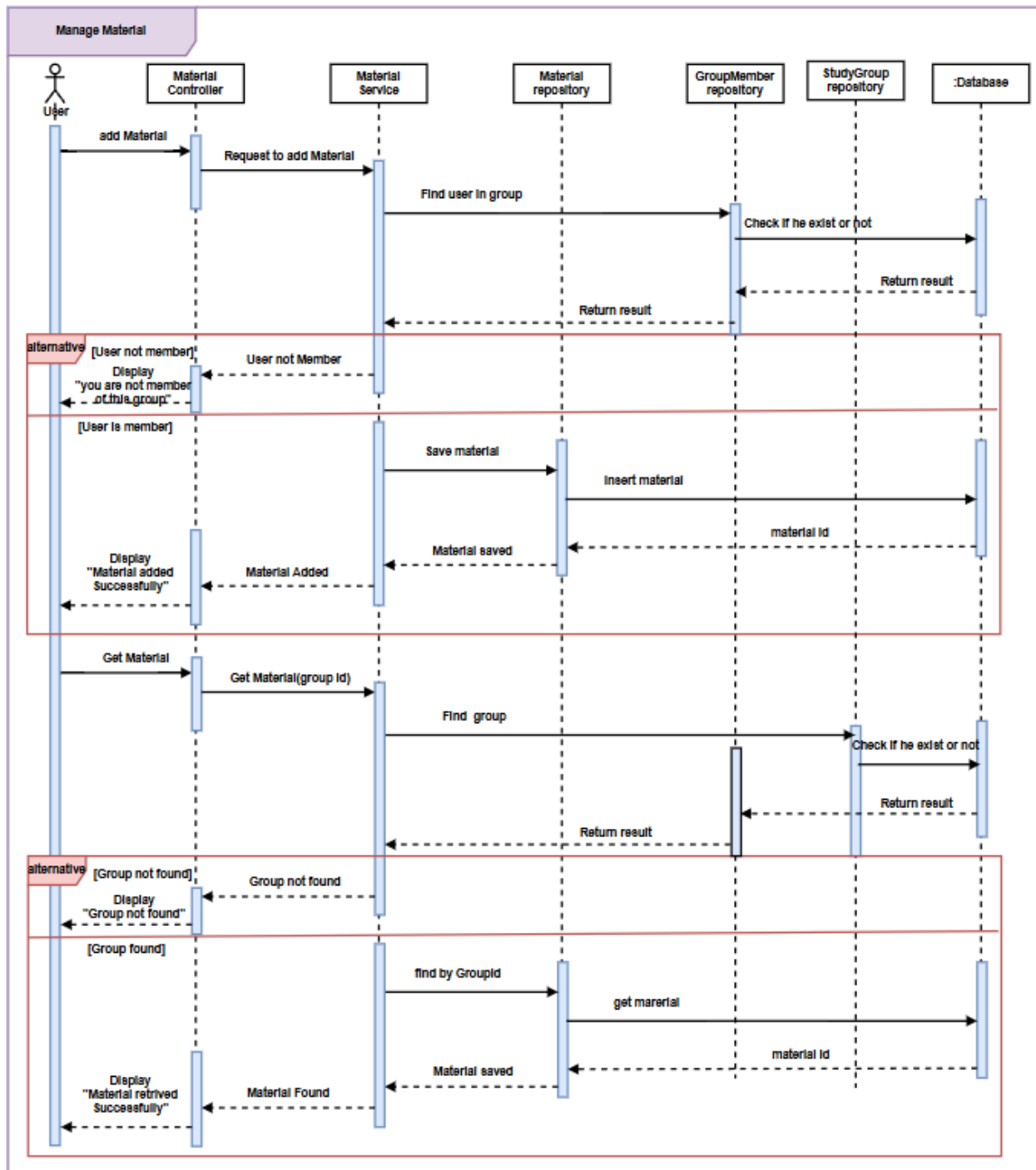
Activity Diagram

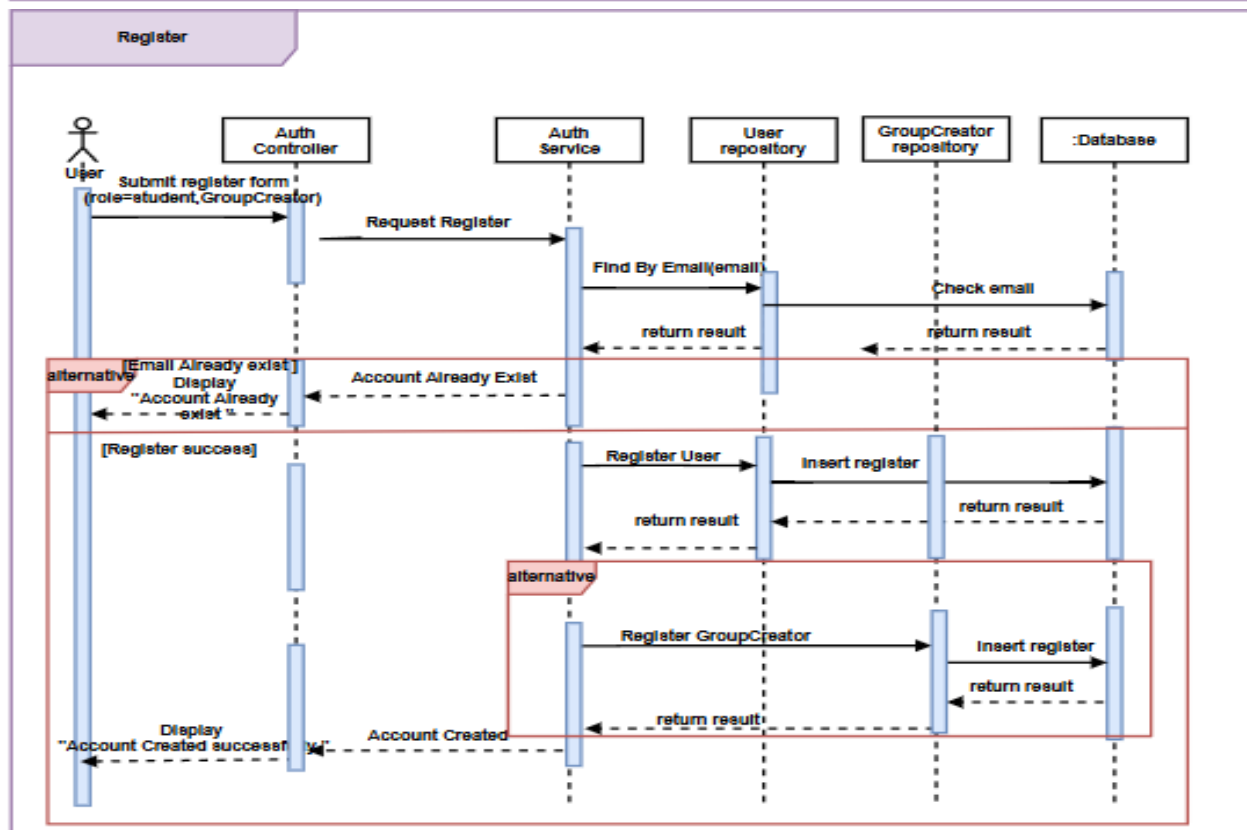
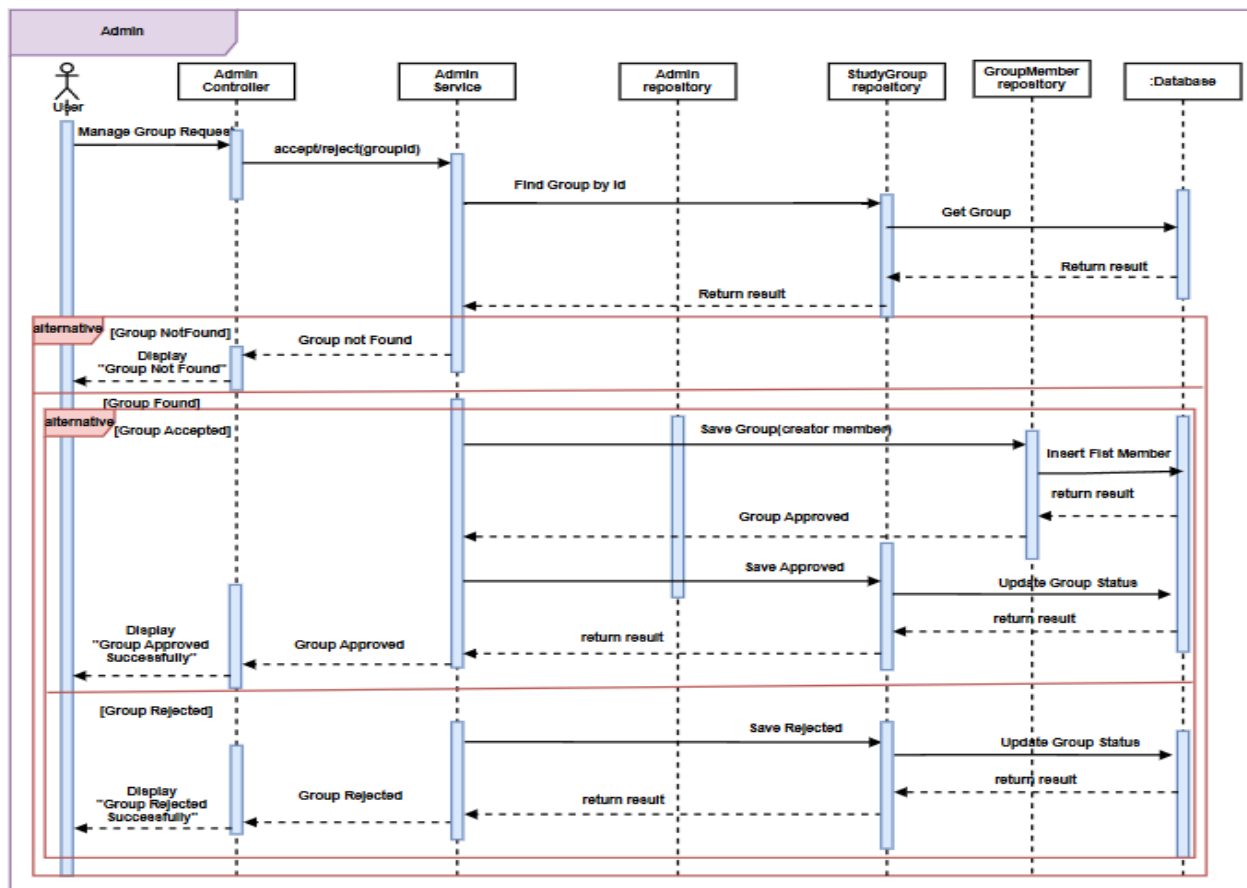
Update-Delete

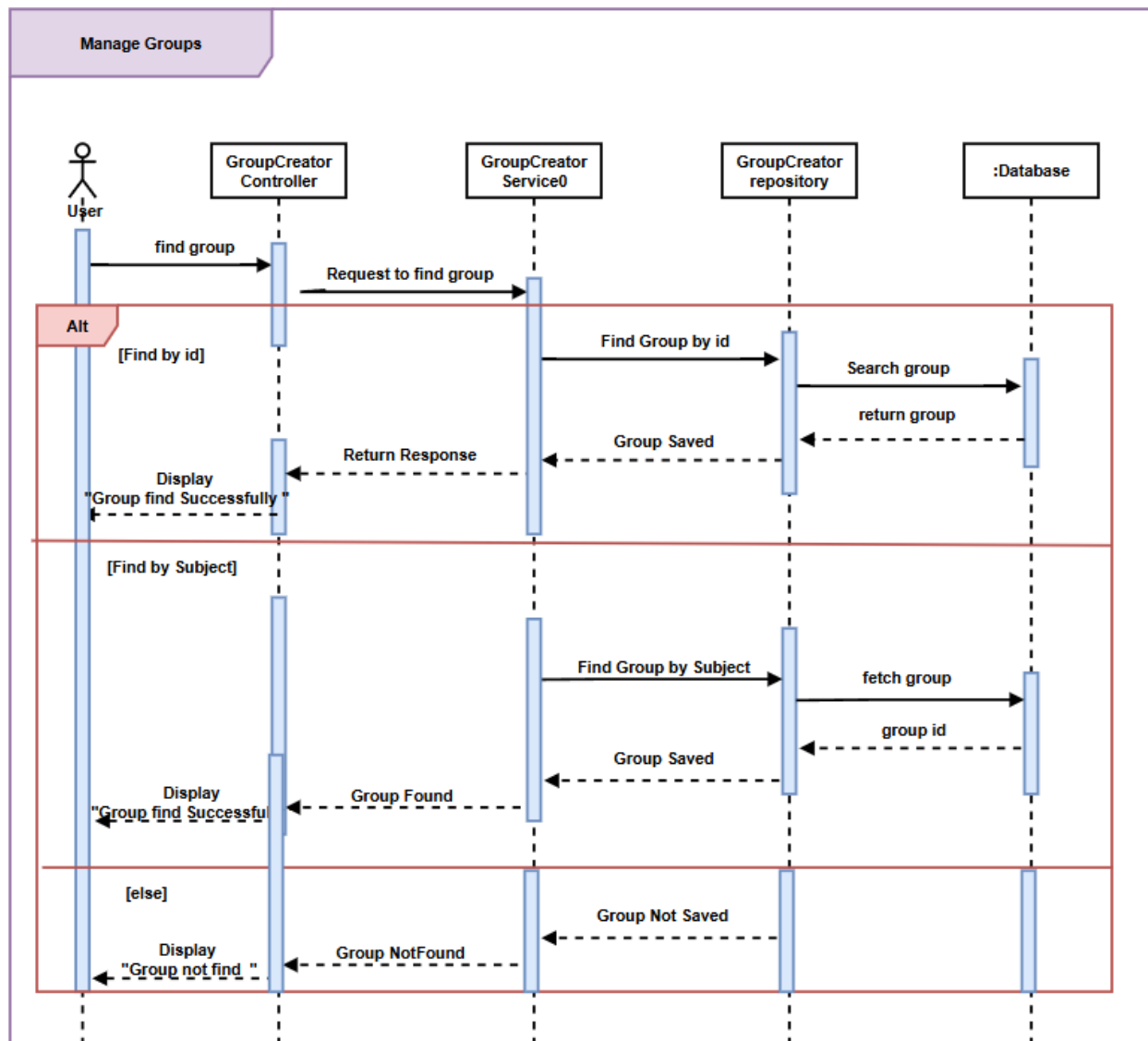
StudyGroup



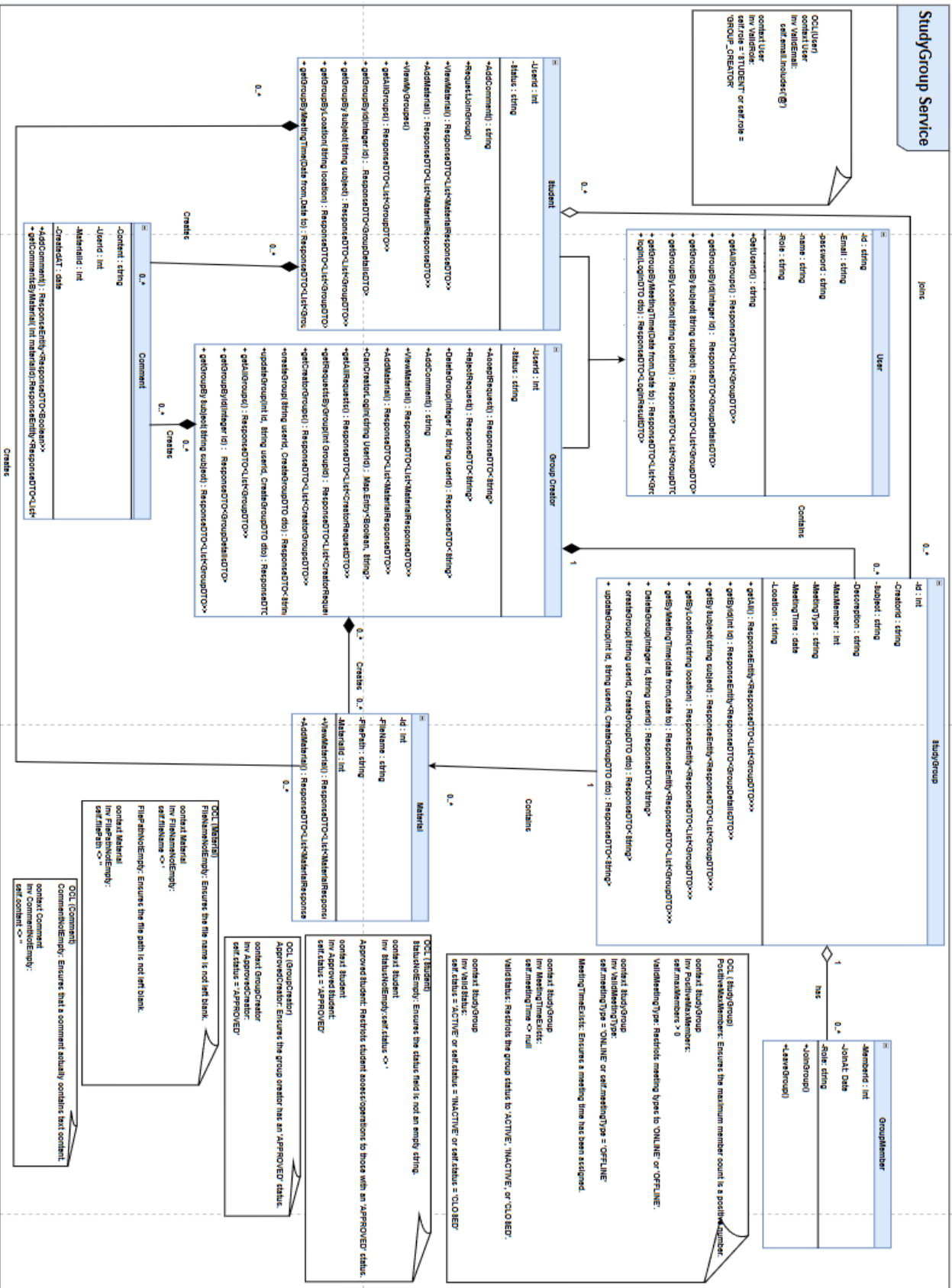
Sequence Diagrams:







Class Diagram:



Admin Service

OCU(Admin)
context Admin
inv ValidIdUserid:
self Userid <> " and Admin.allInstances().>isUnique(Userid)

context Admin
inv ManageNonAdmins:
self User->toAll() { u role <> ADMIN }

Admin Responsibilities & Logic

Group Approval Logic:
AcceptGroup(string Userid) : Transitions a 'StudyGroup' status from 'PENDING' to 'ACTIVE'.
RejectGroup(string Userid) : Transitions a 'StudyGroup' status to 'CLOSED' or removes the entry.

Creator Verification:
AcceptGroupCreation(string Userid) : Validates a user to allow them to create and manage study groups.
GetAllAccountRequest() : Retrieves a list of users waiting for creator status approval.

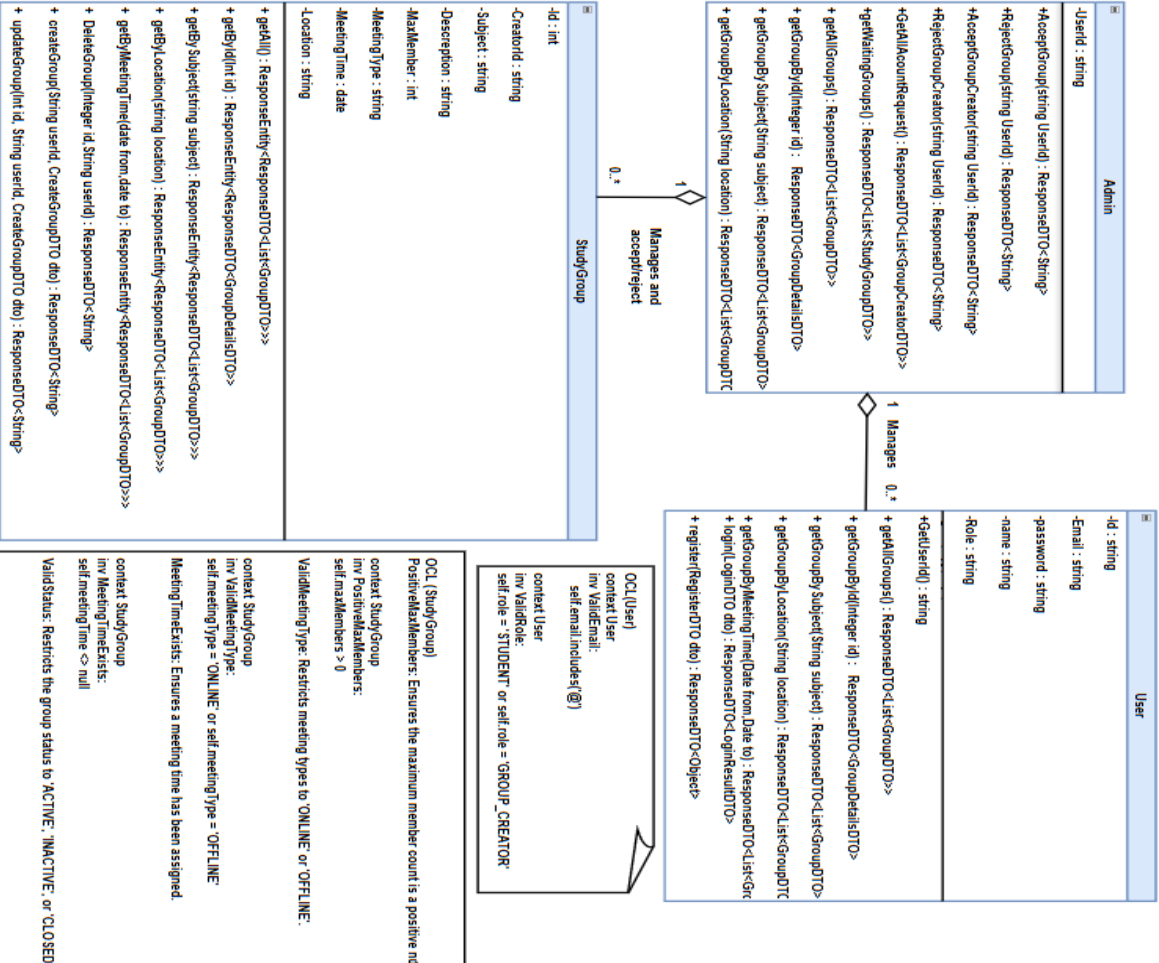
Relationship Constraints

StudyGroup Management: The Admin "Manages and accepts/rejects" "StudyGroup" instances.

context Admin
inv GroupOverSight:
self StudyGroup->size() >= 0 - Admin can oversee any number of groups

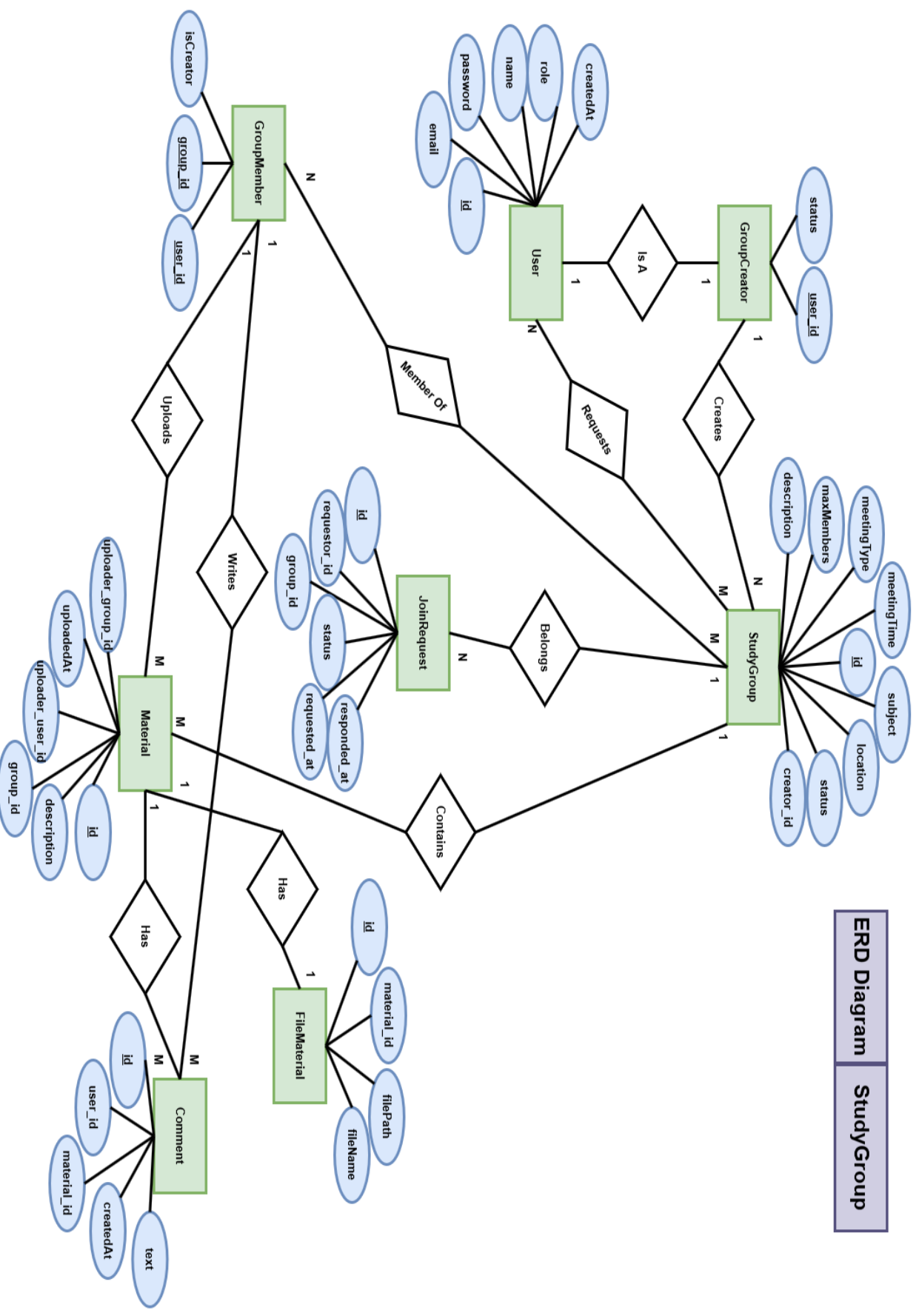
User Management: An Admin is linked to 0 or more users via the "Manages" association.

context Admin
inv UserOverSight:
self User->size() >= 0



ERD Diagram

StudyGroup



Design Patterns Used in the Project

1. Repository Pattern

Where it is used:

Repository layer such as:

- * UserRepository
- * GroupRepository
- * MaterialRepository
- * AdminRepository

Explanation:

Separates database operations from business logic.

Example:

```
```java
userRepository.findByEmail(...)
```
```

2. Service Layer Pattern

Where it is used:

Service classes such as:

- * AuthService
- * GroupService
- * StudentService
- * AdminService

Explanation:

Handles business logic between Controller and Repository layers.

Example:

```
```java
Controller → Service → Repository
```
```

3. Singleton Pattern

Where it is used:

Spring Beans:

- * @Service
- * @Component
- * @Configuration

Examples:

- * JwtService
- * JwtAuthFilter
- * SecurityConfig

Explanation:

Spring creates one shared instance of these classes during application runtime.

4. Dependency Injection (IoC Pattern)

Where it is used:

Constructor injection using:

- * @RequiredArgsConstructor

- * @Autowired

Example:

```
```java
private final JwtService jwtService;
```
```

Explanation:

Objects are injected automatically by Spring instead of creating them manually.

5. DTO Pattern

Where it is used:

DTO package:

- * LoginDTO

- * GroupDTO

- * ResponseDTO

- * MaterialResponseDTO

Explanation:

Transfers only needed data between backend and frontend.

Example:

```
```java
GroupDTO dto = new GroupDTO();
```
```

6. Strategy Pattern

Where it is used:

Spring Security password encoding.

Example:

```
```java
@Bean
public PasswordEncoder passwordEncoder() {
 return new BCryptPasswordEncoder();
}
```

```

Explanation:

Different password encoding algorithms can be changed without modifying application logic.

7. Decorator Pattern

Where it is used:

Security filters.

Example:

```
```java
public class JwtAuthFilter extends OncePerRequestFilter
```
```

Explanation:

Adds authentication behavior to request processing dynamically.

8. Proxy Pattern

Where it is used:

Spring AOP and transactional methods.

Example:

```
```java
@Transactional
public ResponseDTO<String> acceptGroup(...)
```
```

Explanation:

Spring creates proxy objects to add transactions, logging, and security automatically.

9. Factory Pattern

Where it is used:

Spring Bean creation.

Example:

```
```java
@Bean
public PasswordEncoder passwordEncoder()
```
```

Explanation:

Spring acts as a factory that creates and manages application objects.

10. Builder Pattern

Where it is used:

JWT token generation.

Example:

```
```java
Jwts.builder()
```
```

Explanation:

Builds JWT token step-by-step.

11. Adapter-like Pattern

Where it is used:

DTO mapping inside services.

Example:

```
```java
GroupDTO dto = new GroupDTO();
dto.setld(group.getld());
```
```

Explanation:

Converts Entity objects into API response objects.

12. Chain of Responsibility Pattern

Where it is used:

Spring Security filter chain.

Examples:

- * JwtAuthFilter
- * UsernamePasswordAuthenticationFilter

Explanation:

Request passes through multiple filters sequentially.

13. Facade Pattern

Where it is used:

Service layer.

Examples:

- * AuthService
- * AdminService

Explanation:

Services provide simple methods that hide complex internal operations.

Example:

```
```java
authService.login(...)
```
```

14. Microservices Architecture Pattern

Where it is used:

Whole system architecture.

Components:

- * StudyGroup Microservice

- * Admin Microservice

Explanation:

Application is divided into independent services.

15. API Gateway Pattern

Where it is used:

API Gateway service.

Explanation:

Acts as a single entry point for client requests.

16. Service Discovery Pattern

Where it is used:

Discovery Server.

Explanation:

Allows microservices to register and discover each other dynamically.

Functional Requirements

FR-1 User Registration

Description:

The system shall allow users to register accounts as Student or Creator using name, email, password, and role.

FR-2 User Login

Description:

The system shall allow registered users to log in using email and password and receive a JWT authentication token.

FR-3 JWT Authentication

Description:

The system shall authenticate secured requests using JWT tokens.

FR-4 Creator Approval

Description:

The system shall allow the Admin to approve or reject creator account requests.

FR-5 Group Creation

Description:

The system shall allow approved creators to create study groups.

FR-6 Group Approval

Description:

The system shall allow the Admin to approve or reject created study groups.

FR-7 View All Groups

Description:

The system shall allow users to view all approved study groups.

FR-8 View Group Details

Description:

The system shall allow users to view detailed information about a specific study group.

FR-9 Search Groups

Description:

The system shall allow users to search study groups by:

- * subject
- * location
- * meeting time

FR-10 Update Group

Description:

The system shall allow group creators to update their study groups.

FR-11 Delete Group

Description:

The system shall allow group creators to delete their study groups.

FR-12 Send Join Request

Description:

The system shall allow students to send requests to join study groups.

FR-13 View Student Requests

Description:

The system shall allow students to view all their join requests and request statuses.

FR-14 Accept Join Requests

Description:

The system shall allow creators to accept student join requests.

FR-15 Reject Join Requests

Description:

The system shall allow creators to reject student join requests.

FR-16 View Creator Requests

Description:

The system shall allow creators to view pending join requests for their groups.

FR-17 View Student Groups

Description:

The system shall allow students to view groups they joined successfully.

FR-18 View Creator Groups

Description:

The system shall allow creators to view groups they created.

FR-19 Upload Study Materials

Description:

The system shall allow group members to upload study materials and files to their groups.

FR-20 View Study Materials

Description:

The system shall allow group members to view uploaded study materials.

FR-21 Add Comments

Description:

The system shall allow group members to add comments on study materials.

FR-22 View Comments

Description:

The system shall allow users to view comments related to study materials.

FR-23 Role-Based Authorization

Description:

The system shall restrict access to endpoints based on user roles such as Student, Creator, and Admin.

FR-24 Secure Password Storage

Description:

The system shall encrypt user passwords before storing them in the database.

FR-25 API Gateway Routing

Description:

The system shall route client requests through the API Gateway to the correct microservice.

FR-26 Service Discovery

Description:

The system shall allow microservices to register and discover each other dynamically using the Discovery Server.

FR-27 Unified API Responses

Description:

The system shall return responses using a unified response structure through ResponseDTO.

Clean Code Application

1. Layered Architecture

The system is organized into separate layers:

- * Controller Layer
- * Service Layer
- * Repository Layer
- * DTO Layer
- * Security Layer

This separation improves maintainability and reduces coupling between components.

2. Meaningful Naming Conventions

Classes, methods, and variables use clear and descriptive names.

Examples:

- * AuthService
- * GroupService
- * JwtAuthFilter
- * requestToJoinGroup()
- * getAllCommentsByMaterialId()

This improves readability and understanding of the code.

3. Single Responsibility Principle (SRP)

Each class handles one specific responsibility.

Examples:

- * JwtService handles JWT operations
- * CommentService handles comment management
- * MaterialService handles material management
- * AdminService handles admin operations

4. Dependency Injection

Spring Dependency Injection is used through constructor injection and `@RequiredArgsConstructor`.

Example:

```
private final JwtService jwtService;
```

This reduces tight coupling and improves testability.

5. Reusability and Modularity

Interfaces are used to make services reusable and modular.

Examples:

- * `IGroupService`

- * `ICommentService`

- * `ICurrentUserService`

6. DTO Usage

DTO classes are used to transfer data between layers without exposing database entities directly.

Examples:

- * `LoginDTO`

- * `GroupDTO`

- * `ResponseDTO`

This improves security and data abstraction.

7. Unified API Response Structure

The project uses a consistent response format through `ResponseDTO<T>`.

Example:

```
ResponseDTO<Boolean>
```

```
ResponseDTO<List<GroupDTO>>
```

This simplifies API handling and frontend integration.

8. Validation and Error Handling

Business rules and validations are handled inside service classes.

Examples:

- * checking existing emails during registration

- * validating user permissions

- * validating request status before processing

9. Avoiding Code Duplication

Shared functionality is centralized into reusable services.

Example:

`CurrentUserService` is reused for retrieving authenticated user information instead of duplicating authentication logic.

10. Organized Project Structure

The project is organized into packages:

- * Controller
- * Service
- * Repository
- * Models
- * DTO
- * Security
- * Config
- * Enums

This improves code navigation and maintainability.

11. Transaction Management

@Transactional is used in operations requiring database consistency.

Examples:

- * acceptGroup()
- * acceptRequest()

12. Security Best Practices

The project applies secure coding practices:

- * password encryption using BCryptPasswordEncoder
- * JWT authentication
- * role-based authorization using Spring Security